Moca — Template Onboarding Guide

Last modified: 2026-11-09T04:26:40Z

> Hand-maintained. Bump the line above when you edit this guide.

This guide takes a container image to a **sandbox template**: a named image that a client, or an agent session, asks for by id, and whose commands then run in microVMs restored from that image's snapshot. It walks through two worked examples end to end: Moca's public sandbox image (anyone can reproduce it) and a SWE-bench instance (the benchmark case it was built for). It was validated on 2026-10-29 and 2026-10-30 against an SBX0 deployment: the control plane and relay on OpenShift, and the workers on a bare-metal KVM host. Since then the same flow has run self-serve through the API for the SWE-bench Lite instances on templates, and for two kinds of client (a stock agent and Moca Pi) on one control plane and one build, with every provenance link exact (2026-11-07, §8).

- **A template is one worker serving one snapshot.** Onboarding is: build a snapshot from the image, check it in a real microVM, give it a worker, register it, use it, and retire it when a new image replaces it.
- **Everything for templates is in this folder:** the tools, the two sample specs, their tests, and links to the related files elsewhere.
- The short reference for the same steps is `deploy/sbx/template-onboarding/ONBOARDING.md`. Deploying SBX0 itself is `deploy/sbx/README.md` and `RUNBOOK.md`; using a sandbox is the user guide; templates in Moca's own agent sessions are designed in `2026-10-30-session-sandbox-templates-design.md`.
- **Every host and registry below is a placeholder:** `sbx-api.example.com`, `registry.example.com`. No real host, account id or token appears here.

In a hurry? §2 is the whole flow on one page; §4 is the public-image example as copy-pasteable commands; §6 is a client choosing between the two sample templates.

Contents

- 1. Mental model (read this first)
  - What a snapshot holds
  - Where things run
  - Names
- 2. The flow on one page
- 3. The template spec
  - The probe must be shareable
- 4. Example 1: Moca's public sandbox image
  - 4.1 Write the spec
  - 4.2 Build and probe on the KVM host
  - 4.3 Register (operator machine)
  - 4.4 Install the worker
  - 4.5 Use it
- 4b. From an OCI image to a Moca template (self-serve)

    - 4b.1 What you need, and what you get
    - 4b.2 Pick your path: the profile
    - 4b.3 Make your image Moca-ready: the Moca minimum
    - 4b.4 Path A: `base`, worked with `python:3.12-slim`
    - 4b.5 Path B: your own gate record (`general-self`, `benchmark-self`)
    - 4b.6 What Moca records, and your sandboxes' network
    - 4b.7 When something's refused (what users hit)
- 5. Example 2: a SWE-bench instance (django__django-11133)
    - 5.1 The spec comes from image-factory
    - 5.2 Build, register, install
    - 5.3 A client: seed first
    - 5.4 An agent: OpenCode on the template
- 6. Two templates side by side: the client chooses
- 7. Update and retire
- 8. Templates in Moca agent sessions
    - A session pinned to a build, end to end
- 9. Capacity
- 10. Troubleshooting (what we actually hit)
- 11. Regenerating the documents

---

1. Mental model (read this first)

1. **A sandbox runs every command in a fresh microVM** restored from its template's golden snapshot, with the sandbox's own `/workspace` attached. Files in `/workspace` persist across commands; nothing else does.
2. **One worker (`microvm-worker`) serves one snapshot.** A template is a name for a worker, so a new image of a template is a new snapshot and a new worker; the old one drains.
3. **The client chooses the template** when it creates a sandbox (`mocactl sbx create --template ID`, `POST /v1/sandboxes {template}`), and a sandbox keeps its worker for life.

What a snapshot holds

Every template is its own bootable image: there is no common Moca image underneath. What templates share is Moca's thin boot layer, which `build-snapshot.sh` copies into each snapshot.

In a snapshot	Shared by every template?	Size (a 4 GiB SWE-bench guest)
`kernel`	yes: the host's one guest kernel (`--kernel`)	~28 MB
`agent`	yes: Moca's guest agent, from `remote-worker`	~3 MB
`/sbin/init`	yes: Moca's init (mounts, `/dev/fd`, `/var` overlay, `HOME`), inside the rootfs	a few KB

In a snapshot	Shared by every template?	Size (a 4 GiB SWE-bench guest)
`rootfs`	no: an ext4 image of the template's container image (OS, toolchain, repository, `/var`)	3.5-9 GB
`memfile`, `vmstate`	no: the guest's RAM and CPU state after it booted	the guest's RAM (4 GiB), a few KB

So each template costs 7-14 GB of disk, mostly its rootfs and memfile, and disk is what limits how many templates a host holds (§9). Within one template every VM is restored from the same files by hardlink and gets only its own workspace disk, so more sandboxes of a template cost little.

Where things run

Component	Runs on	Does
`template-build.sh`	the KVM host, as root	image → snapshot, checked in a microVM → a template record
`template-register.sh`	the operator's machine	record → SBX0's inputs (`SBX_WORKERS`, the templates file)
`setup.sh`	the operator's machine, against the cluster	mints the worker's token and bundle; restarts the control plane only
`install-host.sh`	the KVM host, as root	installs the worker, which attaches to the relay
`mocactl sbx`	anywhere	lists templates, creates sandboxes, runs commands

Names

For a template id `ID` and an image digest `sha256:abcdef012345…`:

What	Name
snapshot	`ID-abcdef012345` (the first 12 hex of the digest)
worker id	`sbx_<ID with - as _>_abcdef012345`
record	`$SBX_SRV/templates/<snapshot>.json`

2. The flow on one page

#	Where	Command	Check
1	anywhere	write `spec.json` (§3)	`template-build.sh` validates every field first
2	KVM host	`template-build.sh spec.json --kernel K [--registry H]`	digest, `/etc/hosts`, the probe in a microVM; a failure publishes nothing
3	operator	`template-register.sh --env-file F record.json`	one worker per template; an update keeps the old worker
4	operator	`setup.sh --env-file F`	the control plane rolls; the relay does not
5	KVM host	`install-host.sh BUNDLE --snapshot S --budget-mb N [--no-egress]`	the worker attaches

#	Where	Command	Check
6	client	`mocactl sbx create --template ID`	the sandbox runs on that template's image
7	both	new image: steps 1-5 again, then `--retire` the old worker	its live sandboxes have ended

3. The template spec

A JSON file, schema `moca-sbx-template-spec/1`. Every field is checked before anything runs (`template-build.sh`):

Field	Meaning
`id`	what clients ask for: `^[a-z0-9][a-z0-9-]{0,30}$`; the same across versions
`image`	`repository@sha256:<digest>`, never a tag; a repository with no host takes it from `--registry`
`guest_ram_mb`, `vcpus`	the guest's size, fixed in the snapshot
`guest_home`	optional, a writable `HOME` under `/tmp` (the rootfs is read-only)
`skip_git_identity_check`	`true` for an image that refuses agent commits on purpose
`guest_home_gitconfig`	optional, `true`: copy the image's `/root/.gitconfig` into `guest_home`; refused with `skip_git_identity_check`
`egress`	`false`: no network at all (`--no-egress`); `true`: the host's allowlisted egress
`seed`	a bash command that fills a new sandbox's empty `/workspace`, or `""`
`probe`	`{cmd, timeout_s, expect_rc, expect[]}`: run after the seed in the restored VM; passes on the exit code and every regex
`description`, `tools`	what clients see in `GET /v1/templates`
`etc_hosts_sha256`	optional: the build stops unless the rootfs's `/etc/hosts` has this hash

Which images a self-serve create accepts is the deployment's policy: `allowedRegistries` (by registry host; `<default>` is a reference with no host, which the host pulls from its configured registry), and, per registry, `allowedRepositories` naming the repositories it admits (exact path segments, or `prefix/*`). Docker Hub's key is `docker.io` (`index.docker.io` normalizes to it), so write a Docker Hub image with its host, `docker.io/library/python@sha256:…`. Don't work around a refusal by writing another host for the same registry (`registry-1.docker.io/…`): that host is its own key, refused unless an admin lists it, and listing it would bypass the repository allowlist.

Which profile a create names is its gate: the policy's profiles, listed by `GET /v1/policy`. `base` needs no gate record. `benchmark` needs a gate record that passed and that an admin trusts (`trustedGateRecords`). Two self-attested profiles (register G41) accept the creating subject's own gate record, for that subject's templates only, which stay private until an admin trusts the record: `benchmark-self` (the benchmark rules, offline) and `general-self` (no X11, so the KVM host's allowlisted egress). A build shows `attestation: deployment | self`. A profile exists on a deployment only once its stored policy lists it, so check `GET /v1/policy` first. See the subject trust spec.

The probe must be shareable

The probe proves the image's toolchain works in a microVM, not that a task is solvable. For a benchmark, **use a public test target, never the tests that grade a fix**: their names and counts are answer-key material. Match the runner's output as it prints it (§10 has two ways this went wrong).

4. Example 1: Moca's public sandbox image

Moca's own sandbox image, `ghcr.io/rossoctl/moca-remote-worker`, is public, so this example needs no registry key. It is the image SBX0's `default` template ran on 2026-10-29 (built then with `build-snapshot.sh` directly). On 2026-10-31 this exact spec was built with `template-build.sh` (with `--egress-ca`, for egress credentials) and its probe passed in the restored VM.

4.1 Write the spec

This spec is `examples/default.json` in this folder:

```
{
  "schema": "moca-sbx-template-spec/1",
  "id": "default",
  "image": "ghcr.io/rossoctl/moca-remote-
   worker@sha256:eed901b838bf11e16054fccfb38af600a6e61d74dd52907af0750bba13ecbed3",
  "guest_ram_mb": 256,
  "vcpus": 1,
  "egress": true,
  "seed": "",
  "probe": {
    "cmd": "git --version && python3 -c 'print(6*7)' && rg --version | head -1",
    "timeout_s": 120,
    "expect_rc": 0,
    "expect": ["^git version ", "^42$", "^ripgrep "]
  },
  "description": "MOCA's sandbox image: bash, git, Python 3, ripgrep",
  "tools": ["bash", "git", "python3", "rg"]
}
```

`egress: true` gives the guest the host's allowlisted egress (GitHub by default, RUNBOOK). Use `false` for a template that must stay offline.

4.2 Build and probe on the KVM host

```
sudo deploy/sbx/template-onboarding/template-build.sh deploy/sbx/template-
   onboarding/examples/default.json --kernel /path/to/vmlinux
```

Expected output, ending:

```
build-snapshot.sh: verify command exited 0; the last of its output:
...
template-build: the probe passed
==> built template default: snapshot default-eed901b838bf, worker sbx_default_eed901b838bf
==> record: /srv/sbx/templates/default-eed901b838bf.json (copy it to the operator's machine for
   deploy/sbx/template-onboarding/template-register.sh)
==> then, after setup.sh: sudo deploy/sbx/install-host.sh <bundle of sbx_default_eed901b838bf> -
   -snapshot default-eed901b838bf --budget-mb <at least 1888>
```

A 256 MiB guest's snapshot is small; a 4 GiB one is about 7.3 GB (§9).

4.3 Register (operator machine)

Copy the record over, then:

```
deploy/sbx/template-onboarding/template-register.sh --env-file <env file> default-
   eed901b838bf.json --apply
```

It adds the worker to `SBX_WORKERS` and the template to the templates file, then runs `setup.sh`. Only the control plane restarts, so running commands on other templates are not interrupted (on 2026-10-30 the relay's settings hash stayed `1cf8f5032ae16280` across two template changes).

4.4 Install the worker

Copy the bundle `deploy/sbx/.generated/workers/sbx_default_eed901b838bf` to the host by path (it holds a token: never print it), then:

```
sudo deploy/sbx/install-host.sh <bundle dir> --snapshot default-eed901b838bf --budget-mb 8192
```

Expected: `sbx-worker@sbx_default_eed901b838bf attached to the relay`. The worker first checks the snapshot against its manifest and against the template record `template-build.sh` left in `$SBX_SRV/templates/` (`SH_TEMPLATE_RECORD`); a mismatch stops it at start, naming the field. The record's id also becomes the worker's `SANDBOX_TEMPLATE`, so Moca agent sessions naming the template find it (section 8).

4.5 Use it

```
mocactl sbx templates
mocactl sbx create --template default --name try-default
mocactl sbx run -- python3 -c 'print(6*7)'
```

4b. From an OCI image to a Moca template (self-serve)

For a **general user**: you have a Moca login and an image, and no operator access. Every step goes through the API (`POST /v1/templates`) and `mocactl sbx`. Moca builds and probes the template on a KVM host and registers it by policy; nobody approves it. Sections 4 and 5 are the operator's path for the same result.

The commands that go through the API are as recorded in the validation runs, cited by run and request (answers abridged; their ids replaced by placeholders). The self-serve steps for **your own** image are *(unverified)*: they haven't yet been run end to end on a KVM host from a user's image.

4b.1 What you need, and what you get

- **An image, pinned by digest** (`repository@sha256:<64 hex>`), from a registry the deployment's policy allows. Read it first: `GET /v1/policy` lists `allowedRegistries` and, per registry, `allowedRepositories` (§3). Docker Hub is written with its host: `docker.io/library/python@sha256:…` .
- **A Moca login** (`mocactl sbx login`) and room in your quota (`GET /v1/quota`).
- **You get** a template only you can see, built and probed by Moca, which you activate, use from sandboxes or Moca agent sessions, update and retire. Others see it only if you make it `deployment` -visible, which a self-attested template can't be until an admin trusts its gate record (§4b.5).

4b.2 Pick your path: the profile

The profile you name in the create is the gate your template must pass (`GET /v1/policy` lists the deployment's profiles; a profile exists only once its stored policy lists it):

Profile	Gate record	Network	Visible to others	Use it for
`base`	none	the host's allowlisted egress	if you make it so	most templates: a toolchain, an app
`general-self`	**your own**, passed	the host's allowlisted egress	only after an admin trusts the record	a tested image you want proved, with network
`benchmark-self`	**your own**, passed, its floor proved offline	none (X11)	only after an admin trusts the record	a benchmark task environment you gate yourself
`benchmark`	one an admin already trusts (`trustedGateRecords`)	none (X11)	as the deployment's curated templates are	the curated benchmark images

Most users start with `base` (§4b.4). If you want Moca to hold your template to a test result, gate it yourself and use a self-attested profile (§4b.5).

4b.3 Make your image Moca-ready: the Moca minimum

A stranger's image usually fails as published, for reasons most images share:

- **`/etc/hosts` is empty.** A container runtime writes one when a container starts; a microVM guest boots with none, so `localhost` doesn't resolve.
- **The guest agent's minimum is missing**, most often `file` , and any tool your spec's `tools` lists (`git` , `rg` , …).
- **`ca-certificates`** is missing. A template with `egress: true` needs it on a host whose egress intercepts TLS (egress credentials): Moca bakes that host's CA into the system trust store at build, and refuses the build without one. Add it whenever you use egress: it costs little and keeps the template portable across hosts.

The fix is a small layer on top of your image: a localhost `/etc/hosts` , the guest minimum, and `ca-certificates` when the template uses egress. Each path below shows its own. Build it for `linux/amd64` , push it to an allowed registry, and use the digest the registry gives it.

4b.4 Path A: `base`, worked with `python:3.12-slim`

1. **The layer** (a `Containerfile` beside a `hosts` file):

```
# The Moca minimum on python:3.12-slim, for a `base` (or `general-self`) template whose
   tools are bash and
# python3: a localhost /etc/hosts (a microVM guest boots with none), `file` (the guest
   agent's minimum),
# and `ca-certificates` (the system trust store, so Moca can bake a TLS-intercepting
   host's egress CA in
# with update-ca-certificates; on such a host a template with egress true is refused at
   build without it).
FROM docker.io/library/python:3.12-
   slim@sha256:5e2bdb1b0d542d5039cffb8181c65930093949194a65c365dfb1b0fe13a4fa5b
RUN apt-get update -qq && apt-get install -y -qq --no-install-recommends file ca-
   certificates >/dev/null \
 && rm -rf /var/lib/apt/lists/*
COPY hosts /etc/hosts
```

   `hosts`, exactly these two lines (tab-separated; copy them from the rendered guide or the PDF, since the indentation here is the list's):

```
127.0.0.1    localhost
::1  localhost ip6-localhost ip6-loopback
```

   Source: image-factory's `docs/examples/base-python/` (7e5c86b), measured there: a read-only root, no network, every guest-minimum tool present, the probe below printing `(3, 12)`.

2. **The create.** The spec's size must fit the profile's envelope (`base`: up to 16 GiB and 8 vCPUs):

```
{
  "id": "my-python",
  "image": "<your registry>/python-moca-min@sha256:<your layered image's digest>",
  "profile": "base",
  "spec": {
    "guest_mb": 4096,
    "vcpus": 2,
    "egress": true,
    "seed": "",
    "tools": ["bash", "python3"],
    "probe": {
      "cmd": "python3 -c 'import sys; print(sys.version_info[:2])'",
      "timeout_s": 60,
      "expect_rc": 0,
      "expect": ["^\\(3, 12\\)$"]
    },
    "description": "Python 3.12"
  }
}
```

   `POST /v1/templates` with that body and your API token *(unverified for this image)*. A create is answered `202` with the build to follow (as recorded, run lc-1, request 0003):

```
{
  "templateId": "<id>",
  "buildId": null,
  "build": "/v1/templates/<id>/builds/b-<handle>",
  "status": "queued"
}
```

3. **Follow the build** with `mocactl sbx templates --json` or `GET /v1/templates/<id>/builds/<build>`: `queued`, `building`, `probing`, `built`, then `registered` (lc-1/0007 queued, lc-1/0103 registered). `heldBy` names a host once one holds the snapshot.
4. **If it's refused or fails:**
   - **A policy refusal** is `422 policy_refused` naming the rule, before anything is built (lc-1/0001, rule X10). A refused create stores no template, but a `check` record of the refusal stays on the id's stream.
   - **A failed build** has a `reason` with a `step` and a `code` you can branch on, e.g. `{"step": "probe", "code": "probe_failed"}` (lc-1/0185: the probe didn't print what the spec expected). Section 10 lists the usual causes.
5. **Use it:** `mocactl sbx create --template my-python`. Its first use may answer `409 template_not_active` with `Retry-After` (lc-1/0188) while a host starts its worker; the retry succeeds once it's attached. A Moca agent session names it with `--option sandboxTemplate=<id>@<build>` (section 8).
6. **Update and retire:** a new build of the same id (`POST /v1/templates/<id>/builds`), then `mocactl sbx templates tag <id> default --build <new build>`; retire with `mocactl sbx templates retire <id>` (lc-1/0260). A create after that answers `410 template_retired` (lc-1/0262). A retire is refused while a sandbox or session still uses the build (`409 build_in_use`) *(not yet recorded)*.

4b.5 Path B: your own gate record (`general-self`, `benchmark-self`)

A gate record is image-factory's file that says your image's environment works. Sections 4b.5.1-4 are written by image-factory, which owns the tool; 4b.5.5-6 are Moca's side.

1. What a gate checks: the environment, not the answer

A gate record says that your image's **environment works**. The gate runs your image's own test command once, unmodified, as a Moca microVM guest would run it, and asks that the result meet a floor you state. It applies no change, no fix and no patch, and it never sees a task's answer. Whether an agent's work is right is **grading's** question, asked later and elsewhere. A gate cannot answer it, and does not try.

The gate runs your image in an emulation of the guest: a read-only root file system, no network, root, an empty `/workspace` that it seeds from your work tree, and a clean environment (`env -i`, the guest's `PATH`, `HOME=/tmp/home`). In that guest it checks:

Check	Passes when
`/etc/hosts`	your image's own file resolves `localhost`. A guest boots with no runtime-written file, so an empty one fails
tools	the guest agent's minimum (`bash ls cat base64 file`) and every tool your `tools` list names are present

Check	Passes when
no credentials	no credential-shaped variable is in the guest's environment
commits refused	git has no identity, so an agent cannot commit, and a clean tree after the seed if your work tree is a git repository
the floor	your test command's exit code, and its passed count (pytest's or unittest's summary), meet `floor`
absent files	with `absent_sha256`: no file in the image has one of those sha256s (only a count of matches is kept)
the probe	with `probe`: it runs on a fresh seed, and what it prints becomes your template's expectation

2. The input contract (`image-factory/gate-input/1`)

You describe your image in a small JSON file:

Field	Required	Meaning
`schema`	yes	`image-factory/gate-input/1`
`image`	yes	your image, pinned: `repository@sha256:<64 hex>`. A public registry is written with its host (`docker.io/library/python@sha256:…`)
`workdir`	yes	your work tree, an absolute path. The gate copies it into the guest's empty `/workspace`, where the test runs
`test_cmd`	yes	a bash command, run in the work tree
`floor`	yes	`{"rc": N}`, `{"passed_at_least": N}`, or both. `{"rc": 0}` alone is a valid floor
`absent_sha256`	no	sha256s no file in your image may have, such as a task's own tests or its solution
`probe`	no	`{"cmd": "…", "public": true}`: a shareable check, never a grading command
`tools`	no	the tools your template will list (default `bash git python3 rg`)
`network_needed`	no	`true` or absent: your test command cannot pass offline. **`general-self` only**

Any other field is refused, and so is `network_needed` with any value but `true`.

3. Running the gate, and the record it writes

```
python scripts/gate-image.py contract.json --out gate.json \
    --template-id my-app --spec-out spec.json
```

> **Where the tool is.** `gate-image.py` is image-factory's, and today it lives in a repository a general reader can't open. It moves into Moca later. Until then, ask your Moca admin for a copy, or have image-factory gate your image. Everything below is what it does, so you can prepare your image and contract now.

It pulls your image by digest (linux/amd64, as Moca's guests run), and refuses if the digest it got is not the contract's. It writes the record whether or not the gate passed, prints the record's sha256, and exits 0 only on a pass. On a pass, `--template-id` also writes a template spec for the self-attested profiles. Its probe expectation is built from what your probe printed (colour codes stripped), never from a count.

The record, `image-factory/gate/1`, is what you send as `gateRecord` with your create (§4b.5.5). Moca reads these fields:

Field	Moca's rule
`gate.passed`	X1: the gate passed
`image.digest`	X1.image: the record is for your template's image
`observed.testbed.agent_can_commit`, `observed.guest.git_identity`	X12: commits refused (`no` and `none`)
`gate.floor_checked_offline`	X1.offline: `true` means the floor was proved with no network. **`benchmark` and `benchmark-self` require it.** It is `false` only for a `network_needed` contract, which `general-self` accepts
`gate.floor_met_offline`	present only with `network_needed`: whether the offline run met the floor anyway. `general-self` shows it on your build

The record is unsigned bytes, and **its sha256 is its identity**. It names no person or key. Who submitted it is what Moca records, and trust follows from that (§4b.5.6). A record never carries a private registry host, a credential, your test command (only its sha256) or its output (only counts).

4. A worked sample: SWE-bench's django-11133 image, as a stranger would bring it

SWE-bench's own image for django-11133 (`swebench/sweb.eval.x86_64.django_1776_django-11133` , the image behind §5's template), gated as any user's image, outside the benchmark. The contract names its work tree `/testbed` , its own test runner on the public `basic` label, and a floor of rc 0 with the passed count measured. The probe is the same command.

- **As published, the gate refuses it**, correctly, and for reasons most images share: its `/etc/hosts` is empty, and it lacks `file` and `rg` . Its tests themselves pass in the guest, commits are refused, and it holds no credentials.
- **With the Moca minimum on top (§4b.3), it passes.** Here that's a localhost `/etc/hosts` , plus `file` and `ripgrep` from the image's own Ubuntu packages. The gate wrote a spec whose probe expects what the runner printed: `^Ran 53 tests?\b` and `^OK\b` , rc 0. django counts skipped tests in `Ran N` , so an expectation rebuilt from the passed count would never have matched.

The layer, a `Containerfile` beside a `hosts` file:

```
# The Moca minimum, on a stranger's image: a localhost /etc/hosts (a microVM guest boots with
   none)
# and the tools a template needs that the image lacks.
FROM docker.io/swebench/sweb.eval.x86_64.django_1776_django-
   11133@sha256:914093363dbe12def92c5d36e9bdc5270cce5ed0aa6f3daad8bc4ef0389c29bf
RUN apt-get update -qq && apt-get install -y -qq --no-install-recommends file ripgrep >/dev/null
   && rm -rf /var/lib/apt/lists/*
COPY hosts /etc/hosts
```

`hosts` , exactly these two lines (tab-separated):

```
127.0.0.1   localhost
::1 localhost ip6-localhost ip6-loopback
```

The contract, with your layered image's digest in `image` :

```
{
```

```
  "schema": "image-factory/gate-input/1",
  "image": "<your registry>/django-moca-min@sha256:<the digest of your layered image>",
  "workdir": "/testbed",
  "test_cmd": "source /opt/miniconda3/bin/activate && conda activate testbed && export
   LANG=en_US.UTF-8 LANGUAGE=en_US:en LC_ALL=en_US.UTF-8 && ./tests/runtests.py --verbosity 2 --
   settings=test_sqlite --parallel 1 basic",
  "floor": { "rc": 0 },
  "probe": {
    "cmd": "source /opt/miniconda3/bin/activate && conda activate testbed && export
   LANG=en_US.UTF-8 LANGUAGE=en_US:en LC_ALL=en_US.UTF-8 && ./tests/runtests.py --verbosity 2 --
   settings=test_sqlite --parallel 1 basic",
    "public": true
  }
}
```

Add `"passed_at_least"` to `floor` with a count you have measured yourself, if you want the gate to hold the image to it.

Source: image-factory's `docs/examples/general-gate/` and `docs/GENERAL_GATE.md` (8d4b96e). Gated on image-factory's machine; not yet built as a template on a KVM host.

5. Create with your gate record

Send the record's bytes, unchanged, as `gateRecord` with a self-attested profile *(unverified)*:

```
{
  "id": "my-app",
  "image": "<your layered image>@sha256:<digest>",
  "profile": "benchmark-self",
  "gateRecord": "<the contents of gate.json, byte for byte>",
  "spec": { "...": "the spec gate-image.py wrote (spec.json)" }
}
```

- Use `benchmark-self` for a task environment that must stay offline (its record's `gate.floor_checked_offline` must be `true`), or `general-self` for one with the host's allowlisted egress.
- The build shows `attestation: self`, and your template stays private: making it `deployment`-visible answers `409 self_attested`.
- The refusals you may meet: `X1.self.scope` (a record someone else submitted), `X1.offline` (a `network_needed` record on an offline profile), `X12.self` (the record shows commits allowed), and `X1.self.egress` (egress on a profile that forbids it).

6. Sharing it: trust

To let others use it, ask an admin to trust the record: its sha256 goes into the policy's `trustedGateRecords` (`PUT /v1/policy`, a `policy` record names who did it). Its builds then read as `attestation: deployment`, and you can make the template `deployment`-visible. If the admin later removes that trust, the builds read as `self` again and fall back to private; the `policy` record names the builds it narrowed.

4b.6 What Moca records, and your sandboxes' network

- **Provenance:** your create, its gate record, every policy check, the build, each activation, each sandbox or session on it, and its retire are records on the template's stream, chained by digest. Anyone with an export can check them offline (`verify-provenance.py`).

- **Egress records:** for a template with `egress: true`, `mocactl sbx egress <sandbox>` (or `GET /v1/sandboxes/<id>/egress`) shows which hosts the sandbox reached, and the sandbox's end record names the log's digest and whether it's complete. Paths are logged without query, fragment or userinfo, and an injected credential appears by name only, but a secret a guest puts in a URL path segment is logged as sent.

4b.7 When something's refused (what users hit)

You see	Why	Do
`422 policy_refused`, rule `registry`	the image's registry isn't in `allowedRegistries`	use an allowed registry; never another host name for the same registry
`422 policy_refused`, rule `registry.repository`	that registry allows only listed repositories	ask an admin to list yours
`422 policy_refused`, rule `M7.guest`, `M7.vcpus`, …	the spec is outside the profile's envelope	fit the spec to the profile (`GET /v1/policy`)
`429 quota_exceeded`	your templates, queued builds or active templates are at your quota	retire or wait (`GET /v1/quota`)
`409 template_exists`	the id is someone else's, or one the deployment declares (l28g1-1/0006)	another id; an admin can hand a declared id to you (`templates transfer`)
a build failed at `probe`	the probe printed something else, or `localhost` doesn't resolve	the Moca minimum (§4b.3); match what the probe prints exactly
a build refused for egress	`egress: true` without `ca-certificates`	add it to the layer

5. Example 2: a SWE-bench instance (django__django-11133)

This is exactly what ran on 2026-10-30 (the record is in the project's run notes): a SWE-bench Lite instance, built, checked in a microVM, registered, used by a client and by an agent, then retired.

5.1 The spec comes from image-factory

The result, as it ran, is `examples/django__django-11133.json` in this folder (its image names no registry host; give it with `--registry`).

image-factory's moca-microvm records carry the spec under `template`, so nothing is written by hand. Its `probe` is a **public** test target (`basic`), not the instance's grading tests, and its image names no registry host:

```
jq '.template' platforms/moca/microvm/django__django-11133.json > django__django-11133.json
cp platforms/moca/microvm/django__django-11133.json django__django-11133.gate-record.json
```

The second file is the image's **gate record**, unchanged: the build checks the image against it (§5.2). Both are in `examples/`.

The fields that matter: `id: django-django-11133` (the instance id lower-cased, `__` → `-`; a Verified image also in Lite gets `-v`), 4096 MiB and 2 vCPUs, `egress: false`, `seed: "cp -a /opt/testbed-src/. /workspace/"`, `probe.expect: ["^OK\\b"]` with `expect_rc: 0`, and `etc_hosts_sha256`.

5.2 Build, register, install

```
sudo deploy/sbx/template-onboarding/template-build.sh deploy/sbx/template-
   onboarding/examples/django__django-11133.json --kernel /path/to/vmlinux --registry
   registry.example.com --gate-record deploy/sbx/template-onboarding/examples/django__django-
   11133.gate-record.json
```

The spec names its gate record (`source_record_digest`), so the build needs it: before pulling, it checks the gate passed, the record is for this image, and commits are refused, and the template record keeps that evidence (gating rules X10, X12). The run below was on 2026-10-30, before this check existed *(the `--gate-record` form is unverified on a host)*.

As it ran on 2026-10-30 (about 3 minutes, plus the pull):

```
==> etc/hosts as the spec says
build-snapshot.sh: verify command exited 0; the last of its output:
Ran 53 tests in 0.062s
OK (skipped=2)
template-build: the probe passed
==> built template django-django-11133: snapshot django-django-11133-3d3e705ff379, worker
   sbx_django_django_11133_3d3e705ff379
```

Then §4.3 and §4.4 with that record, and **`--no-egress` with `--budget-mb 13408`**: a 4 GiB guest needs (4096 + 32) × 3 + 1024 MiB for one session at standby depth 2.

5.3 A client: seed first

A new sandbox's `/workspace` is empty. **SBX0 does not run the seed: the client does**, as its first command. And use `sbx create`, not `sbx up`: `up` pushes the client's own repository into `/workspace` before the seed runs.

```
id=$(mocactl sbx create --template django-django-11133 --json | jq -r .id)
mocactl sbx run --sandbox "$id" -- bash -c 'cp -a /opt/testbed-src/. /workspace/ && git -C
   /workspace log --oneline -1'
mocactl sbx run --sandbox "$id" -- bash -c 'source /opt/miniconda3/bin/activate && conda
   activate testbed && cd /testbed && ./tests/runtests.py --verbosity 2 --settings=test_sqlite -
   -parallel 1 basic 2>&1 | tail -3'
```

Expected (2026-10-30): the seed in 1-2 s with a clean tree at `bc34802 baseline`, then `Ran 53 tests`, `OK (skipped=2)`. The guest has no network (`connect: Network is unreachable`), and a token for this sandbox is `PERMISSION_DENIED` at any other template's worker.

5.4 An agent: OpenCode on the template

Point the agent's MCP server at the seeded sandbox (`mocactl sbx mcp --sandbox <id>`), keeping `sbx up`'s tools-off block, so the agent works only through the sandbox. On 2026-10-30 a headless `opencode run` with the instance's problem statement used `sandbox_bash`, `sandbox_read` and `sandbox_edit` only, finished in 23 s, and its patch was **graded resolved** by the official SWE-bench harness. Take the patch from the sandbox (`git diff --binary <baseline>` in `/workspace`), not with `sbx pull`, which diffs against the client's repository.

6. Two templates side by side: the client chooses

With both examples onboarded, the deployment serves two templates, each on its own worker. The templates file that `template-register.sh` wrote holds:

```
[
  {
    "id": "default",
    "worker": "sbx_default_eed901b838bf",
    "description": "MOCA's sandbox image: bash, git, Python 3, ripgrep",
    "tools": ["bash", "git", "python3", "rg"]
  },
  {
    "id": "django-django-11133",
    "worker": "sbx_django_django_11133_3d3e705ff379",
    "description": "SWE-bench django__django-11133 (lite) at its baseline bc348023edb4; seed
   with: cp -a /opt/testbed-src/. /workspace/",
    "tools": ["bash", "git", "python3", "rg"]
  }
]
```

A client lists them, then picks one per sandbox. The worker ids never reach the client: it sees only the id, the description and the tools.

```
mocactl sbx templates
```

```
default          MOCA's sandbox image: bash, git, Python 3, ripgrep
django-django-11133 SWE-bench django__django-11133 (lite) at its baseline bc348023edb4; seed
   with: cp -a /opt/testbed-src/. /workspace/
```

(`--json` gives the same list as JSON, with `tools`.)

A sandbox of each, and a command that tells them apart (only the SWE-bench image has the instance's repository at `/opt/testbed-src`):

```
gen=$(mocactl sbx create --template default --name general --json | jq -r .id)
swe=$(mocactl sbx create --template django-django-11133 --name task --json | jq -r .id)
for s in "$gen" "$swe"; do
  mocactl sbx run --sandbox "$s" -- bash -c 'git -C /opt/testbed-src log --oneline -1
   2>/dev/null || echo no-testbed'
done
```

Expected:

```
no-testbed
bc34802 baseline
```

The SWE-bench template also has no network (`egress: false`, checked on 2026-10-30: `connect: Network is unreachable`). `default`'s egress (`egress: true`, through the host's allowlisting proxy, so `git` and `curl` over HTTPS rather than raw TCP) was checked on 2026-10-31 with SBX0's egress credentials: a second user's sandbox cloned, committed and pushed a branch of a private repository with its stored PAT, which never entered the sandbox; another user's sandbox got `401`; deleting the credential revoked it within its cache TTL (`smoke.sh` step 7).

What SBX0 guarantees, as checked live with two templates on 2026-10-29 (`deploy/sbx/templates-check.sh`, 7 checks, 0 failed):

- the list names every template and **no worker id**;
- each create records the template asked for; an **unknown template is refused** (`no such template`) and creates nothing;
- each template's sandboxes go to **its own worker**; two sandboxes of one template share the worker but **not their workspaces**;
- each sandbox's commands run on **its template's image** (the probe saw each image's digest);
- a token for one template's sandbox sent to the other template's worker is `PERMISSION_DENIED` at the relay;
- another user cannot see or use the sandbox;
- `sbx up --template` refuses a repository already bound to a sandbox of another template.

The 2026-10-29 run used `default` and an earlier SWE-bench template, `swe-astropy-14995`; the 2026-10-30 run repeated the choice and isolation checks with `django-django-11133` and `astropy-astropy-14995`. To check a deployment yourself, run `templates-check.sh` against two of its templates (RUNBOOK §14).

7. Update and retire

A new image for a template: a new spec with the new digest, then build, register and install again. The record names a new worker; registering moves the template there and **keeps the old worker** so its live sandboxes keep working. Once none is left:

```
deploy/sbx/template-onboarding/template-register.sh --env-file <env file> --retire <old worker
   id> --apply
sudo deploy/sbx/install-host.sh --uninstall <old worker id>
```

Removing a template entirely is `--remove ID`. On 2026-10-30 retiring `django-django-11133` revoked its worker's token, a create of it was refused (`no such template`), and an astropy sandbox created before kept running.

Self-serve, through the API (a deployment with the template store and host agents; deployed and shown end to end on 2026-11-06): the template's owner does it all, with no operator step.

Point `default` at a new build (new creates use it), then retire the old build, or the whole template:

```
mocactl sbx templates tag django-django-11133 default --build sha256:<new build>
mocactl sbx templates retire django-django-11133 --build sha256:<old build>
mocactl sbx templates retire django-django-11133 --reason 'done'
```

A retire is refused while a sandbox or a session still uses the build (`409 build_in_use`): delete it first. A retired build is never used again (`410 template_retired`), and the host agent then removes its worker and snapshot. Every step is a record naming who did it. The owner also changes the description and visibility (`templates edit`); an admin rejects a build after the fact, transfers a template (`templates transfer`) and sets quotas (`templates quota`).

The host agent's removal keeps each retired build's sandbox and session **workspaces** for the janitor (`install-host.sh --janitor`), which frees them once their sandboxes have ended. On a host without the

janitor they stay on disk: about a few hundred MB to a GB per template used, so a long run of retires fills `/srv/sbx` unless the janitor is installed or the workspaces are removed.

A template the deployment declares (`SH_SANDBOX_TEMPLATES`) has no owner, so a subject cannot create a self-serve template under the same id (`409 template_exists`). An admin hands it to a subject (`mocactl sbx templates transfer ID github:<id>`, register G34); that subject then builds, tags and retires it.

8. Templates in Moca agent sessions

Moca's own agent sessions (Moca Pi, P6) can name a template too: `mocactl run --option sandboxTemplate=ID`. A worker serves a template when `SANDBOX_TEMPLATE` is set; a session with no template never lands on one; the harness runs the seed once, before the agent's first tool call. A templated P4 worker is installed beside the main one with `setup-microvm.sh --template ID --snapshot DIR`; set-up is in `P4-ON-P6.md`. Only one worker per host may use guest egress (moca-egress keys VMs by id, which workers share), so the installer refuses `--egress` beside another worker that uses it. Checked live on a Moca-VM (P6+P4) on 2026-10-31, with a scripted mock model: a session naming `django-django-11133` ran on the templated worker with the repository seeded, its second turn kept the workspace and did not seed again, a session with no template ran on the untemplated worker, and an unknown template was refused at creation.

On the **combined control plane** (one control plane for both, `setup.sh SBX_PROFILE=combined`), a session names a self-serve template the same way, and may pin a build (`ID@sha256:…`): it runs only on that build's workers. Checked on the cluster and the KVM host on 2026-11-05, with no model: a stock-agent sandbox and a session pinned to the same build, and first use through a session (`409 template_not_active`, then created 23 s later).

A session pinned to a build, end to end

On a deployment with the template store (the combined control plane, image `87b02d64` or later):

- **The seed is the build's own.** The harness seeds the session's workspace from the pinned build's `spec.seed`, the same seed a stock agent's client reads for that build, and reports where it came from: `sandbox.seed: {source: "build", buildId: "sha256:…"}` in the turn's result. A client checks that `buildId` equals its pin. (`source: "declared"` is a template from `SH_SANDBOX_TEMPLATES`.)
- **The turn returns the patch.** After each turn on a templated session, the result carries `workspace: {baseline, patch, patchSha256, patchBytes}`: the whole workspace's diff from the commit the seed left (`git diff --cached --binary` on a throwaway index, Moca's own `.moca-*` files left out), also on the terminal `done` and `error` frames, so `mocactl run --json` prints it. It is never truncated: over `SH_TURN_PATCH_MAX_BYTES` (4 MiB by default) `patch` is left out and `patchRefused: "too_large"` says so; a patch that is not UTF-8 comes as `patchBase64`; a failed capture is `{error}` and does not fail the turn. Each turn's patch is the whole end state, so a client takes the last turn's. Extract it with `jq -j` (shell `$(…)` drops the trailing newline, and the checksum then differs).
- **Creating the session for `ID@build`:** today `mocactl run --option sandboxTemplate=` accepts only the deployment's declared ids, and refuses `ID@sha256:…` before sending anything (register G37). Create the session through the API, then run it:

```
curl -sS -X POST -H @<header file with your api token> -H 'Content-Type: application/json'
   --data '{"sandbox": {"template": "django-django-11133@sha256:<build>"}, "model": "
```

```
   <model id>", "credentials": {"inference": "<name>"}}' https://sbx-
   api.example.com/v1/sessions
mocactl run "<prompt>" --session <sessionId> --json
mocactl sessions delete <sessionId>
```

A first create on an inactive template is `409 template_not_active` with `Retry-After`: retry. Without the gateway's host in `SH_MODEL_CATALOG_HOSTS`, the model id is accepted unverified (`modelVerified: false`) and `GET /v1/sessions/{id}` shows it.

- **Provenance:** a `session` record at create (its lease's `activate` record and its build) and one at delete, on the template's stream, as a sandbox has. A session that is never deleted has no end record: delete each session when its work is done (an active pinned session also blocks the build's retire).

Validated on 2026-11-07 (the combined control plane on the cluster, workers on the KVM host, a real model): on four SWE-bench templates built fresh, a stock agent (OpenCode) on a sandbox and Moca Pi on a session, both pinned to the same build. Every session was seeded from its build and returned its patch whole, and an independent composer joined every provenance link of every rep exactly, for both kinds of client.

9. Capacity

Measured on one host (188 GiB, 93 GB free) on 2026-10-31, with SWE-bench templates of 4 GiB guests:

Limit	Rule	Measured
disk, templates	7.3-13.6 GB per snapshot; `template-build.sh` wants `SBX_MIN_FREE_GB` (50 by default) free	8 built in 31 min (140-515 s each, ~70 GB); room for about one more
memory per template	each attached worker keeps its snapshot's memory mapped, idle or not	~4.1 GiB each
disk, sandboxes	each seeded sandbox's workspace	75-240 MB each; 25 sandboxes over 8 templates and 2 users took ~3.5 GB, and disk stopped the test first
commands at once	`--budget-mb` ≥ (guest + 32) × (running VMs) + 1024; a **running** VM is charged its full guest size, a standby its dirty pages	3 per worker at 13408 MiB; every one at 65536 MiB, with almost no real memory used
quotas	`SBX_MAX_PER_SUBJECT`, `SBX_MAX_TOTAL`	deployment settings
logins	a minted login lasts at most 24 h	—

So on a host like this, disk limits templates and sandboxes before memory does, and the default budget allows far fewer commands at once than memory holds: raise `--budget-mb` for parallel runs.

Size the budget for the work run at once on the template, not for one session. `template-build.sh`'s `min_budget_mb`, (guest + 32) × 3 + 1024 (13408 MiB for a 4 GiB guest), fits about two running VMs plus a standby. Two agents working one template at once, each with back-to-back commands, then hit admission refusals (`memory-budget: promoting a standby …`, register G36). Allow one running VM per command that may run at once: **17536 MiB for four 4 GiB VMs**, as the 2026-11-07 runs used with no refusal. The worker's start-up warning (slots × per-VM size against the budget) is expected at any budget below its slot count.

A refused command never ran, and the clients retry it. `mocactl sbx run` and `sbx put` retry an Exec the worker or relay refused before it started (`template_busy`, `busy`), with backoff of 1, 2, 4 and 8 s

(`MOCACTL_SBX_BUSY_RETRIES`, 4 by default, 0 to turn off), and exit **75** when every try is refused. The static client (`remote-worker/cmd/sbx`, and its Python twin) does the same within `SBX_BUSY_RETRY_S` (30 s by default) and logs each refusal. **125** stays for every other sandbox failure, where the command may have run.

10. Troubleshooting (what we actually hit)

Symptom	Cause	Fix
`no match for ^Ran 51 tests?\b`, though the tests passed	an expected count rebuilt from "passed", but Ran counts skips	expect what the runner prints (`^OK\b`, or the Ran total)
`no match for \b8 passed\b`, though pytest passed	pytest coloured its summary (`ESC[32m8 passed`)	fixed in `template-build.sh`: it matches with ANSI stripped
a probe expectation from the gate's counts	the gate counts include grading tests	use a public target (§3)
`git checkout` fails part way as a non-root user	root-owned files from earlier root builds in the checkout	`chown -R` the checkout to its user first; check with `find … -not -user …`
a worker restarts and fails	its env names a snapshot no longer on disk	point it at one that exists before starting
`sbx up` sandbox has the client's repo, not the template's	`up` pushes the client repository	`sbx create`, then the seed (§5.3)
every restore after `vm-9` fails "API socket never appeared"	the jail path overflowed a Unix socket for a 31-character template id	fixed (2026-10-31): jails are keyed by a short hash of the worker id
"no capacity" for minutes, never ending	the session's tier or template has no worker attached	the retry line (and the harness log) now name the cause; attach a worker of it
`memory-budget: promoting a standby would re-price it …`	the worker's `--budget-mb` fits fewer running VMs than the work needs	size the budget for the commands run at once (§9); clients retry, then exit 75
`unknown sandbox template 'ID@sha256:…'; declared: …`	mocactl checks `--option sandboxTemplate` against the declared list	create the session through the API, then `mocactl run --session` (§8)
a session's patch checksum does not match	`$(…)` in the shell dropped the patch's trailing newline	write it with `jq -j '.workspace.patch'` (§8)
a self-serve create is `409 template_exists`, nobody owns it	the id is a template the deployment declares	use another id (§7; register G34)

11. Regenerating the documents

Artifact	Generated by	Source of truth
this guide's table of contents	`python3 docs/tools/gen_toc.py deploy/sbx/template-onboarding/GUIDE.md`	the headings
`deploy/sbx/template-onboarding/GUIDE.pdf`	`python3 docs/tools/gen_pdf.py`	this `.md`

If no generator rebuilds it, don't commit it.